ST2: Tree Data Structures

Binary Search Trees, AVL Trees, Red-Black Trees

- Task 1 - Design and Implementation of Binary Search Tree (BST) Data Structure
  - Console outlog
- Task 2 - BST Tree visualisation and Benchmarking
  - Data Entry
  - Searching Contact
  - Showing all contacts
  - Deleting Entry
  - Running Benchmarker
- Task 3 - Design and Implementation of Balanced Binary Search Trees (AVL and Red-Black Trees)
  - Creating New Entries
  - Searching for Entries
  - Listing all Entries
  - Deleting Values
  - Using rotation operations to balance the tree
- Task 4 - Benchmarking AVL Tree with Binary Search Trees(BSTs)
  - Benchmark Application
  - Benchmark visualisation for sizes: [100, 200, 400, 800, 1600]
  - Benchmark visualisation for sizes: [500, 1000, 2000, 4000, 6000, 8000, 10000]
  - Discussion
    - Insertion
    - Search
    - Deletion
    - Found Count
    - The Effect of Random Data
    - AVL's Height Guarantee
    - Rotation Cost vs Benefit
- Task 5 - Design, Implement, Visualise and Benchmark the Red-Black-Tree
  - Implementation of Red-Black Tree
  - Benchmark BST vs AVL vs Red-Black Tree Benchmark
  - Visualisation of Benchmark

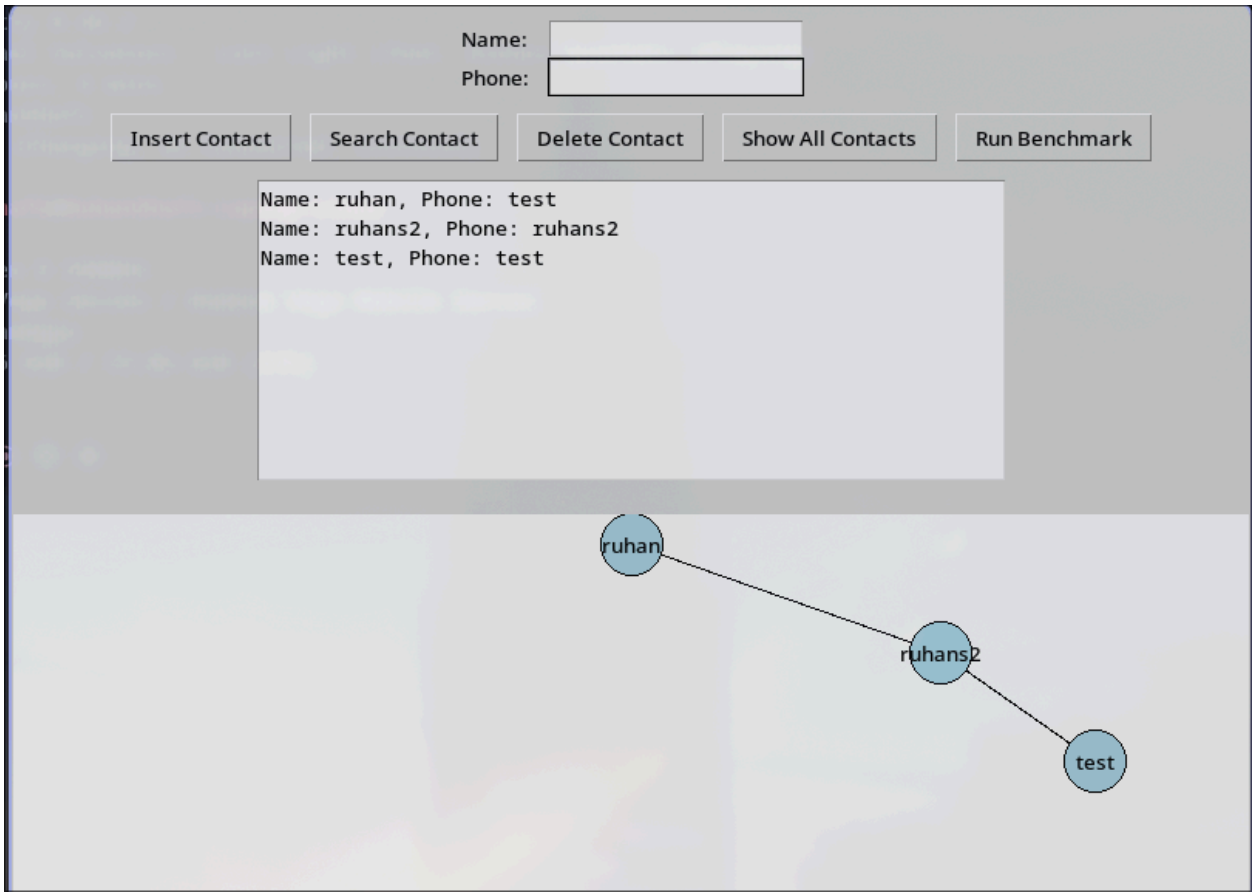
Task 1 - Design and Implementation of Binary Search Tree (BST) Data Structure

Console outlog

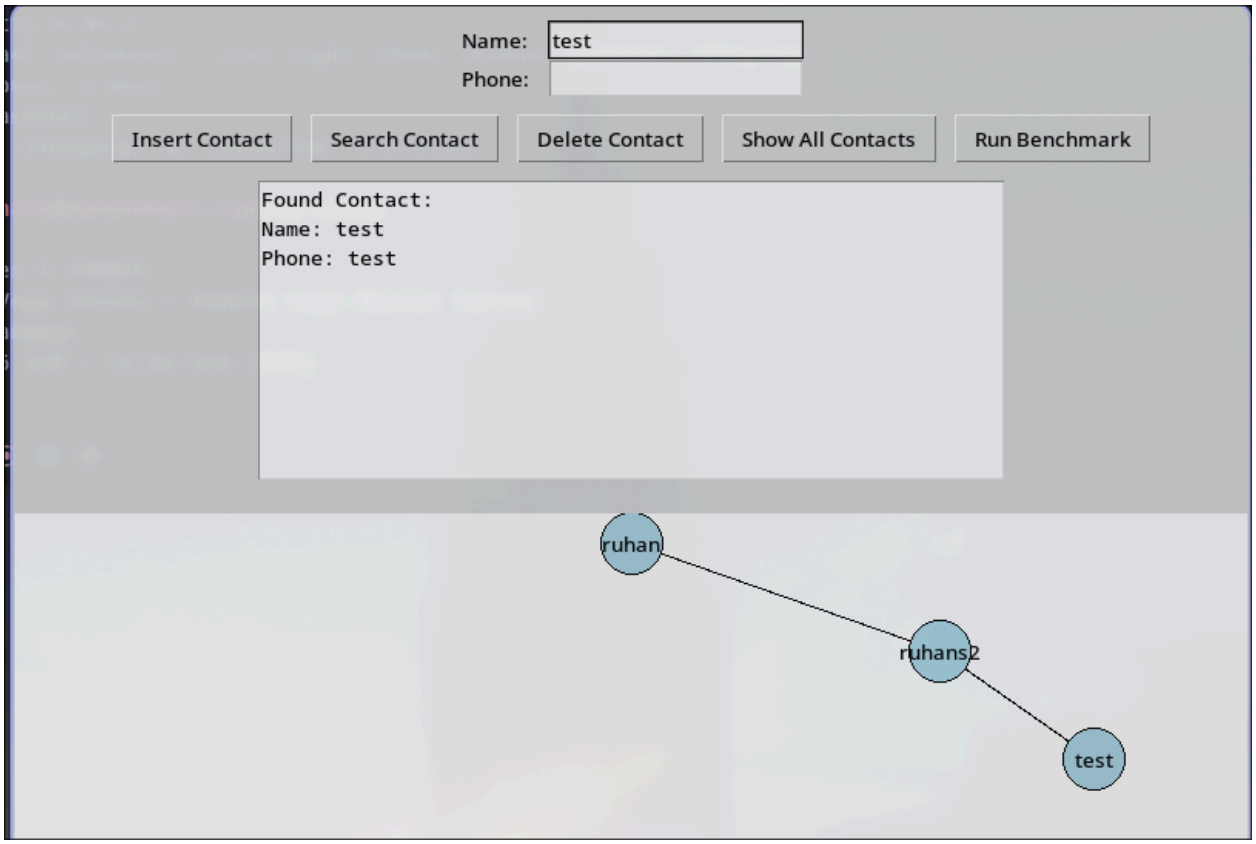
```
└─ python Task1-bst.py
All contacts:
Name: Alice, Phone: 123
Name: Bob, Phone: 234
Name: Eve, Phone: 345

Searching for Bob:
Bob 234

Deleting Alice:
Name: Bob, Phone: 234
Name: Eve, Phone: 345
```

Task 2 - BST Tree visualisation and Benchmarking

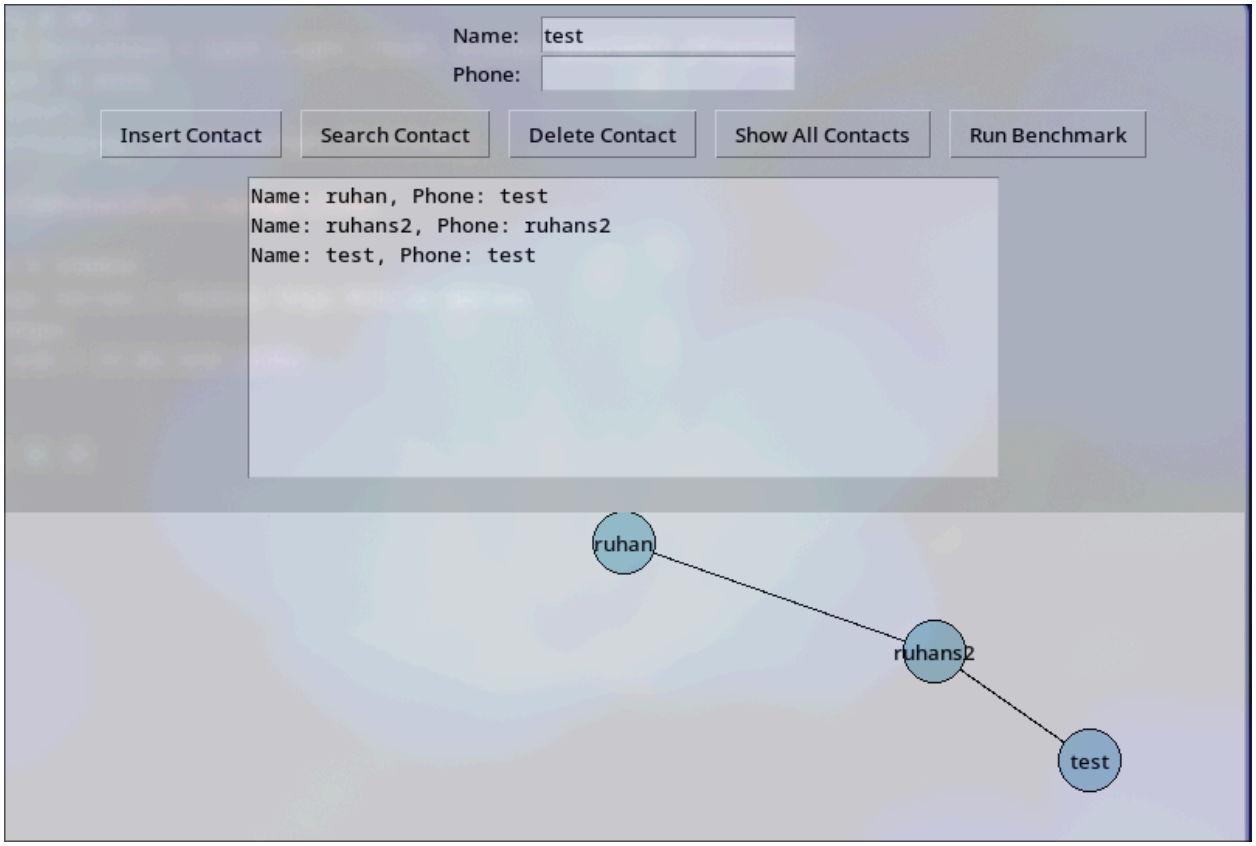
Data Entry



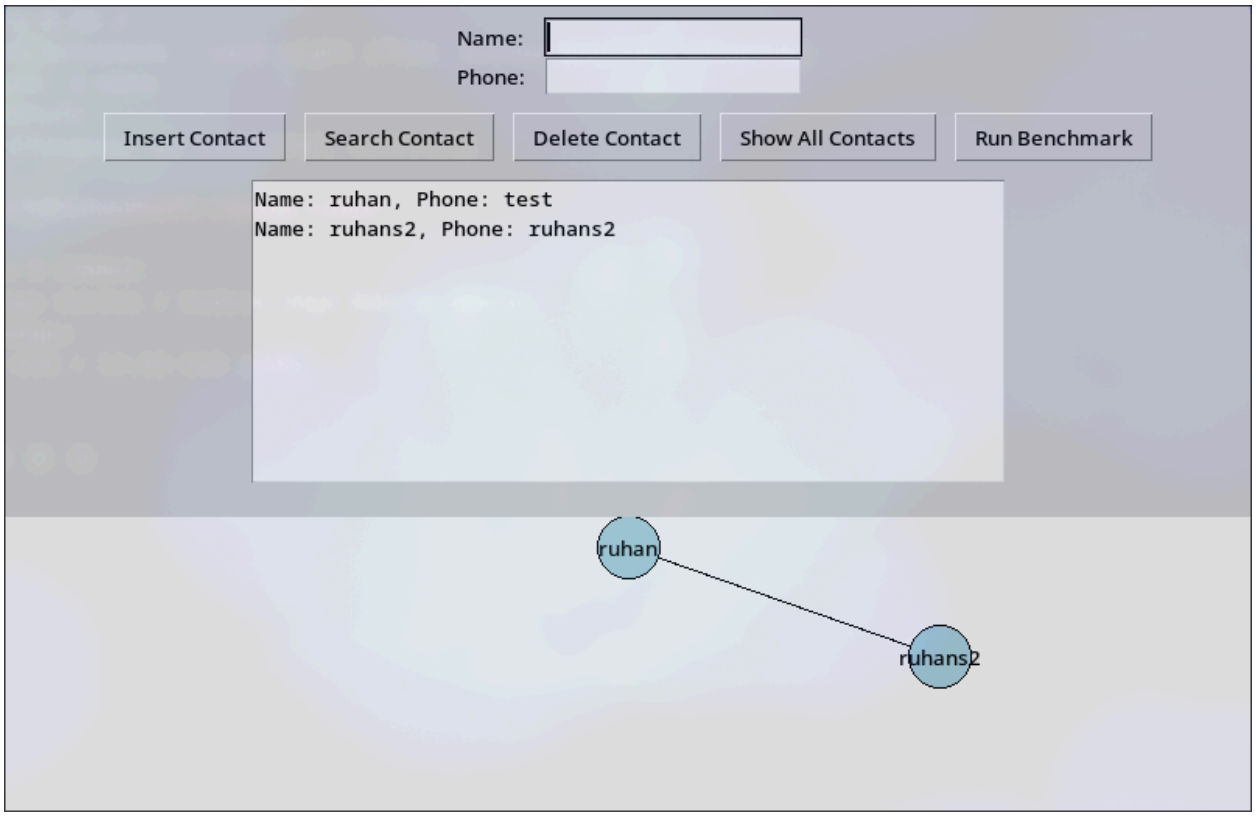
Searching Contact



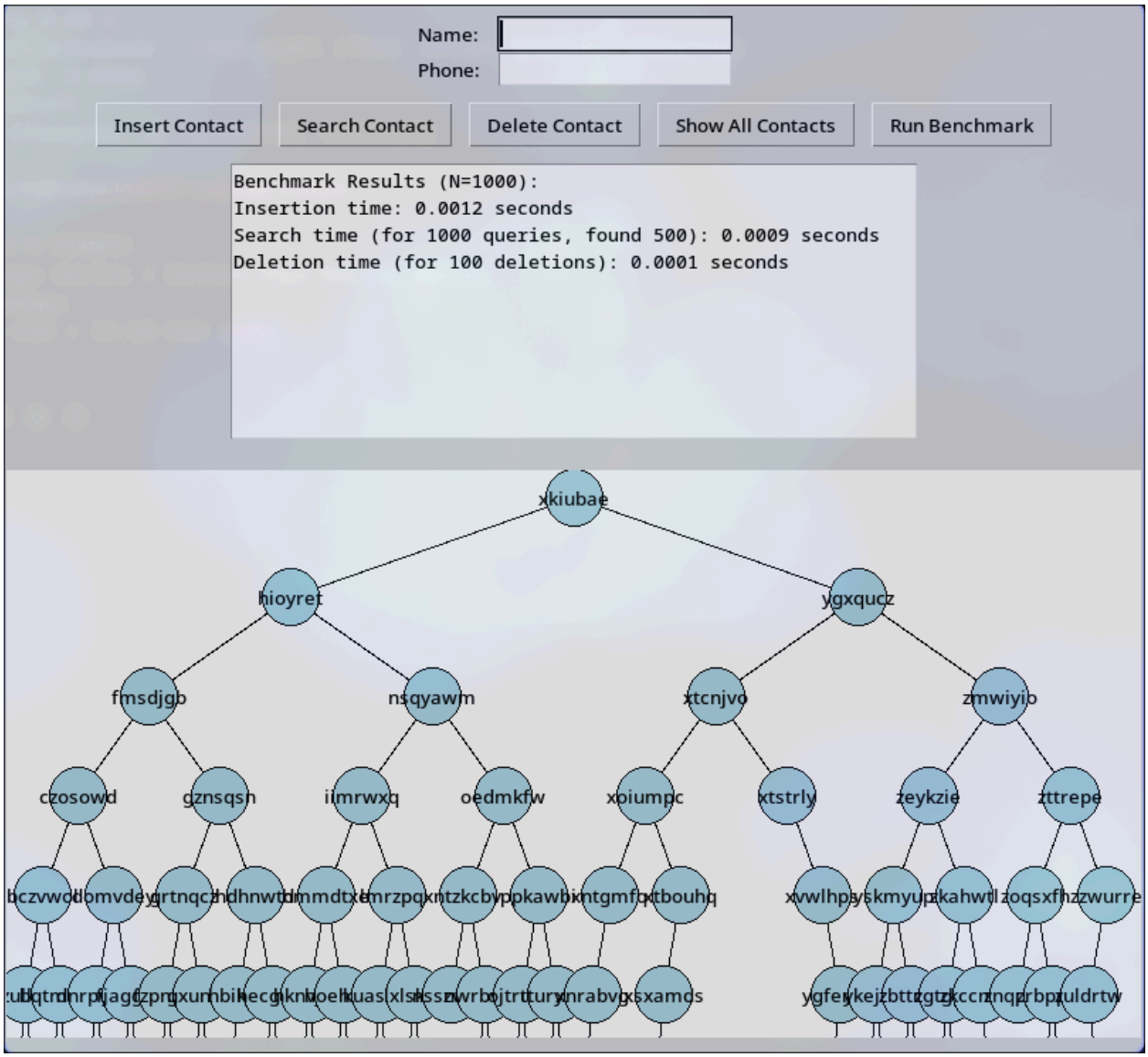
Showing all contacts


Name: test
Phone:
Insert Contact
Search Contact
Delete Contact
Show All Contacts
Run Benchmark
Name: ruhan, Phone: test
Name: ruhans2, Phone: ruhans2
Name: test, Phone: test
ruhan
ruhans2
test

Deleting Entry

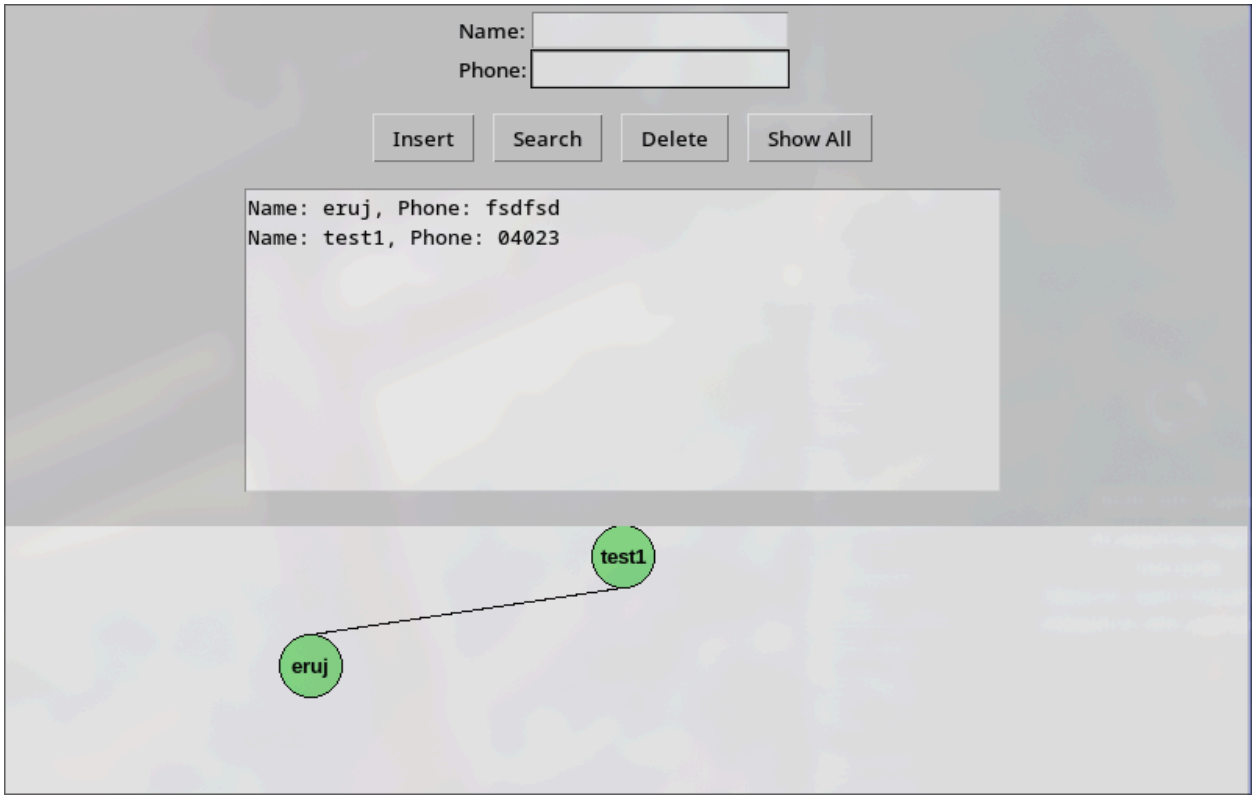


Running Benchmarker

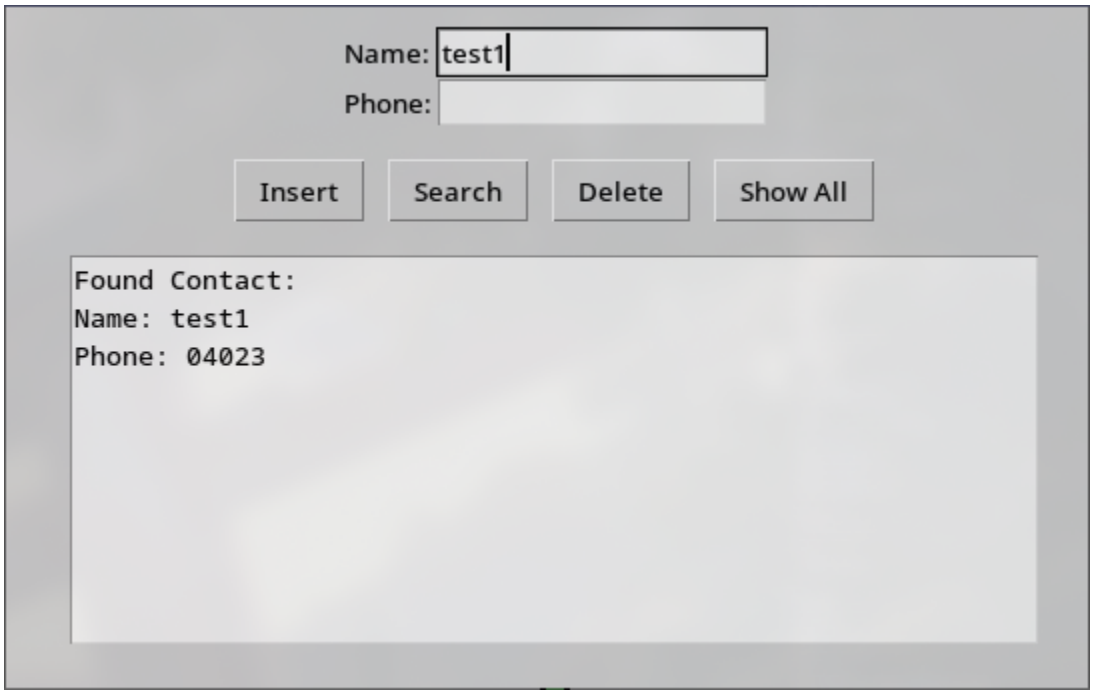


Task 3 - Design and Implementation of Balanced Binary Search Trees (AVL and Red-Black Trees)

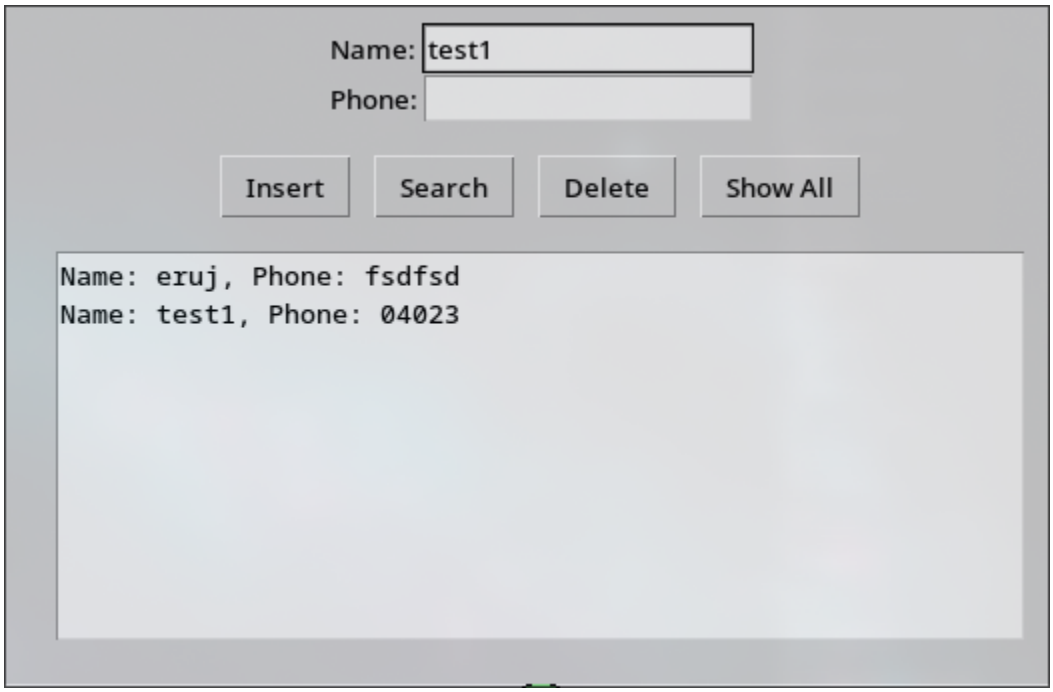
Creating New Entries



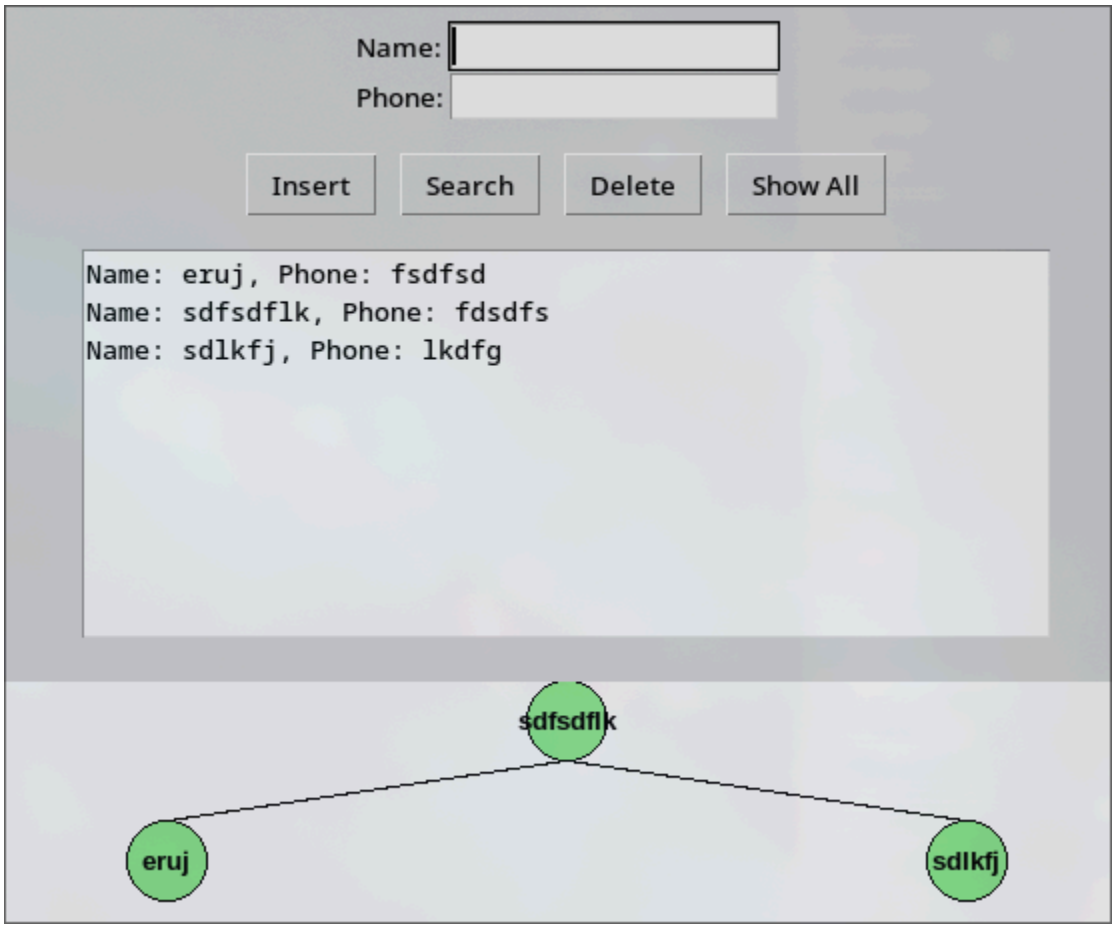
Searching for Entries



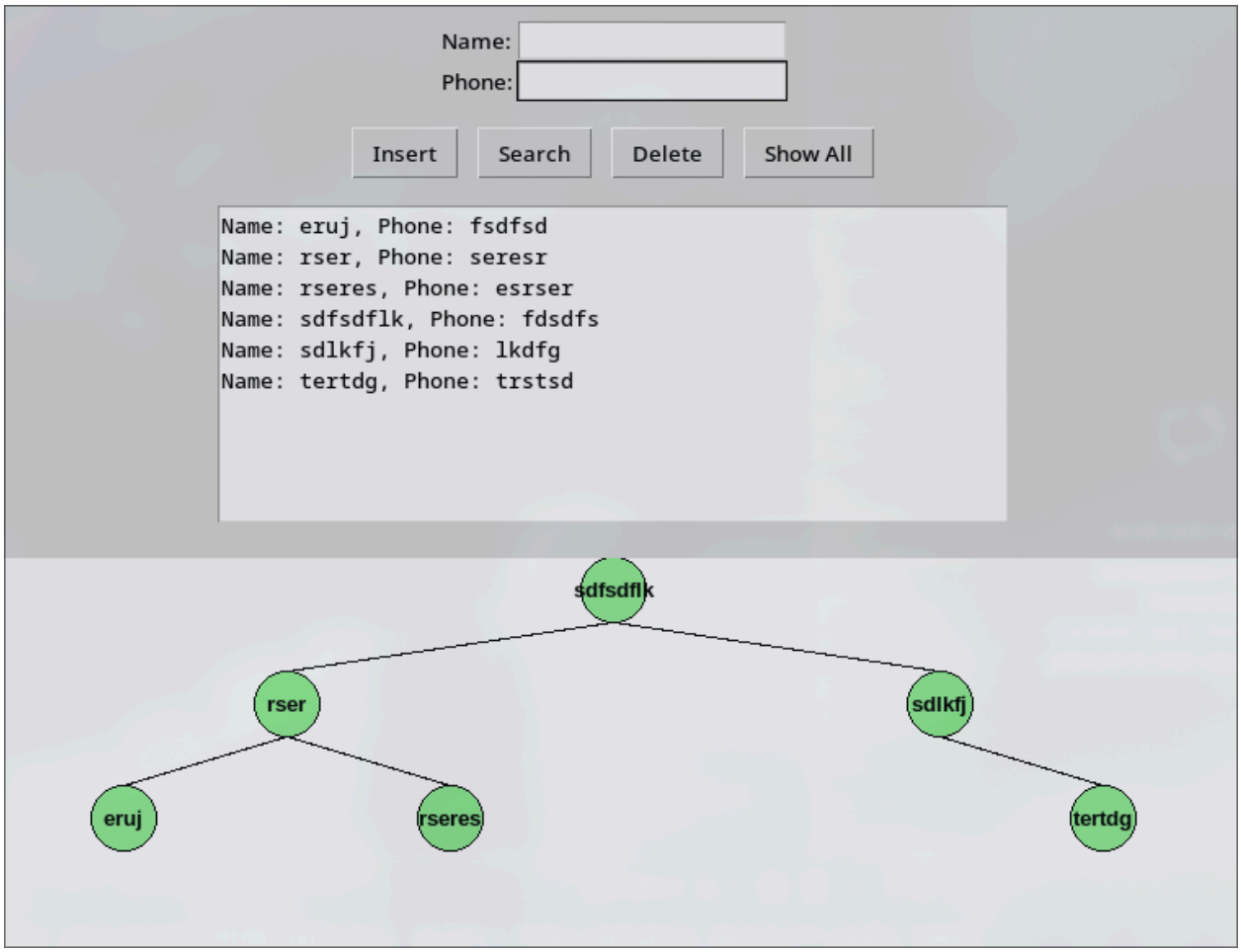
Listing all Entries



Deleting Values

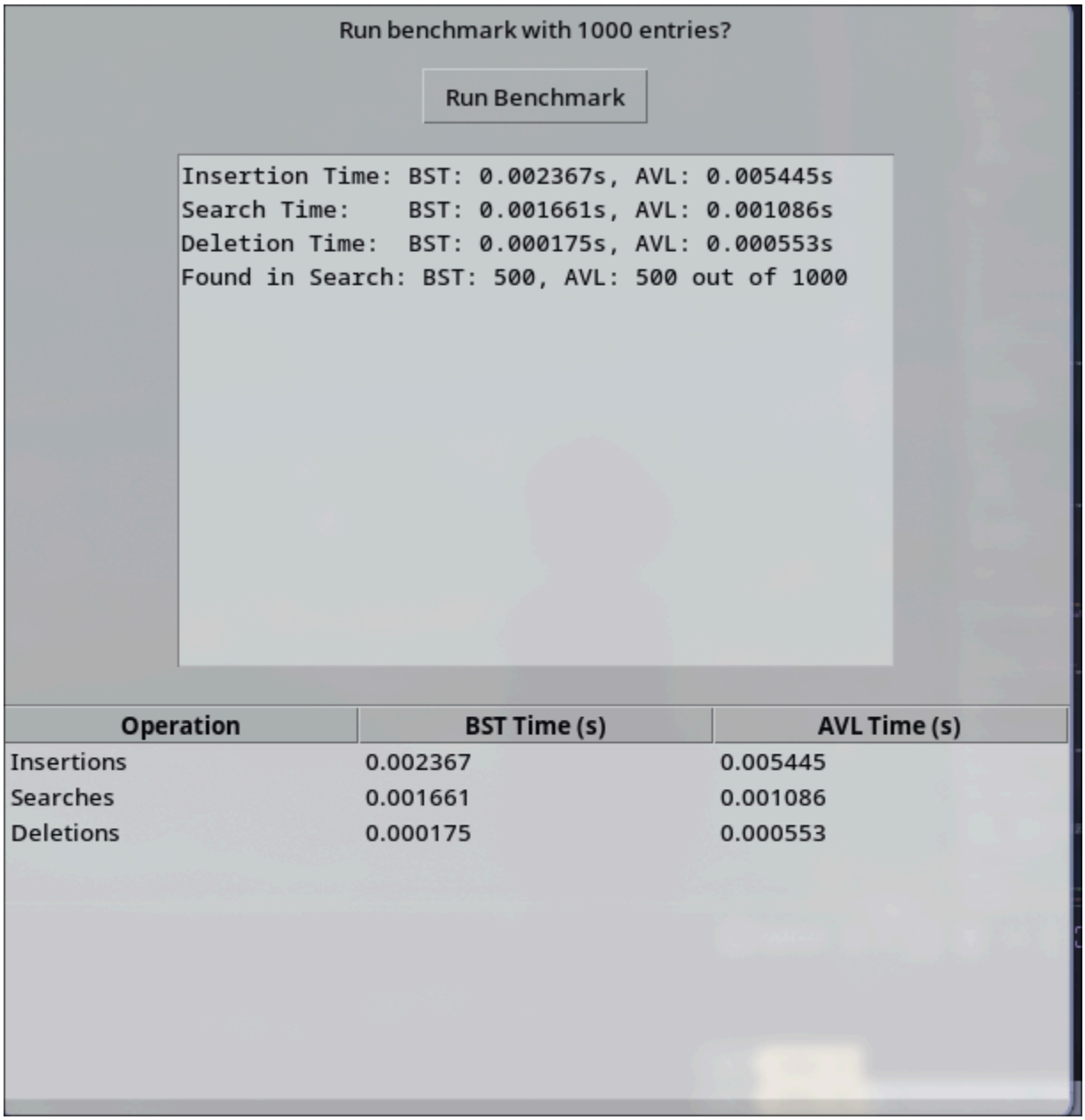


Using rotation operations to balance the tree

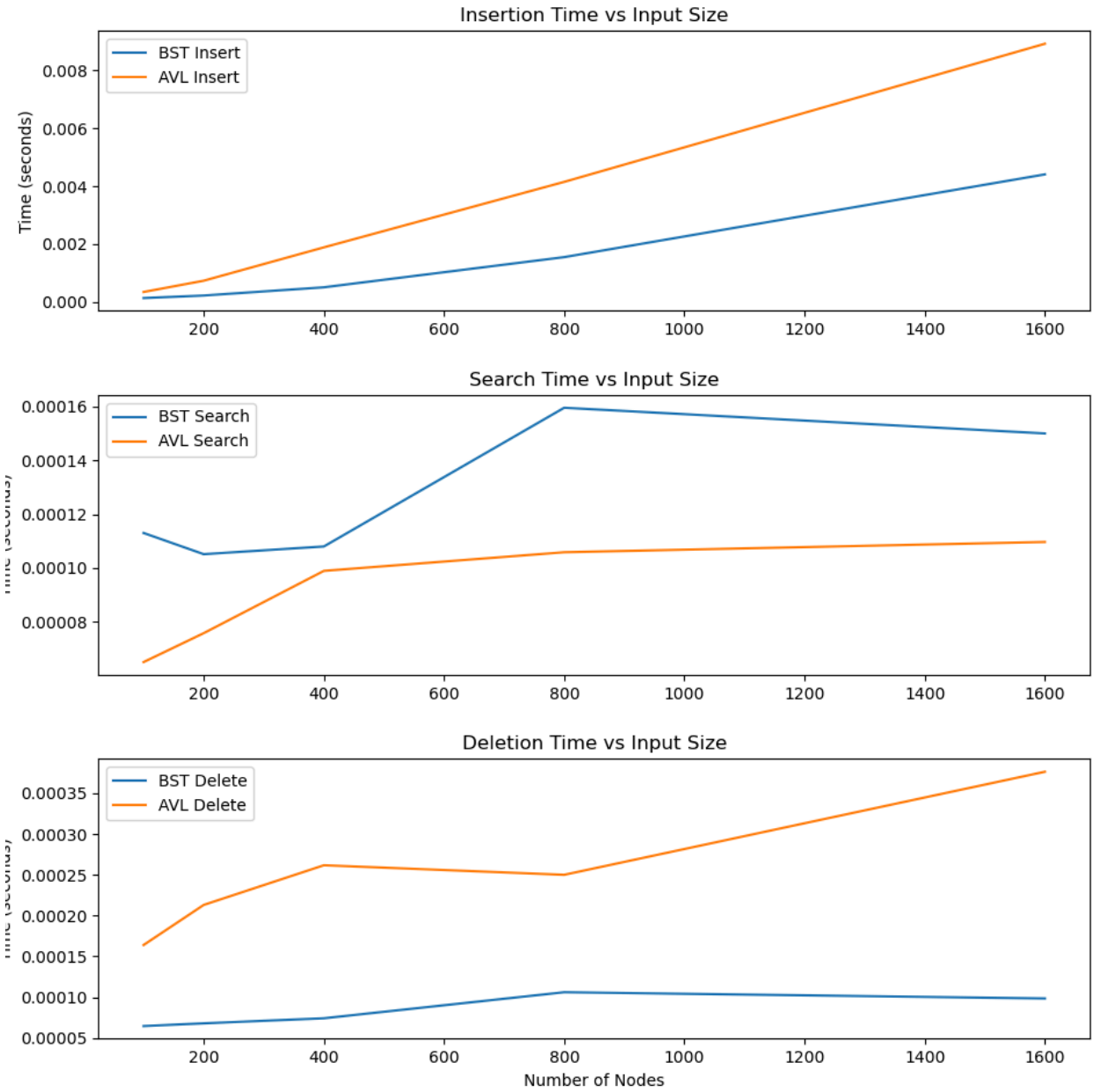


Task 4 - Benchmarking AVL Tree with Binary Search Trees(BSTs)

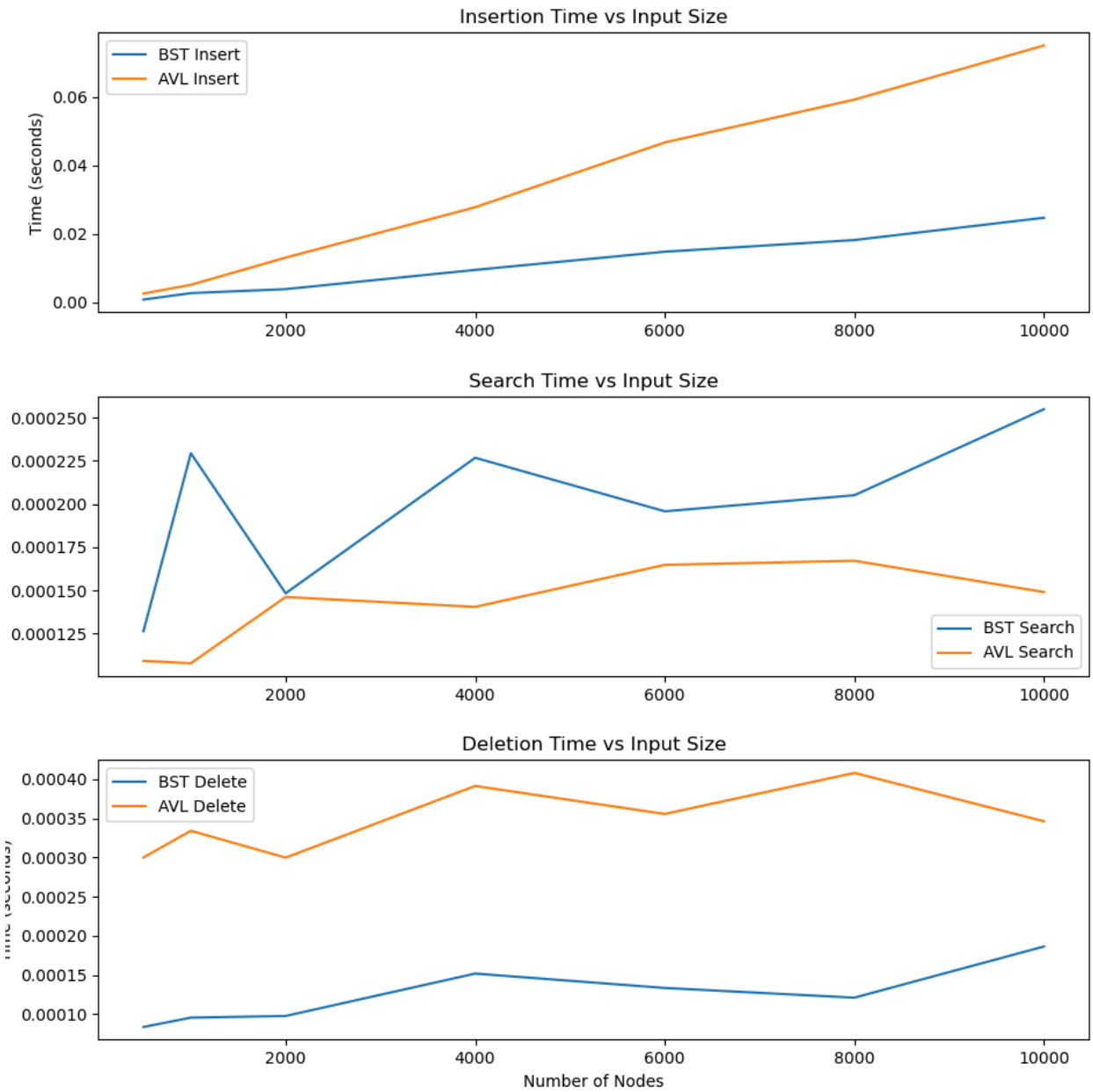
Benchmark Application



Benchmark visualisation for sizes: [100, 200, 400, 800, 1600]



Benchmark visualisation for sizes: [500, 1000, 2000, 4000, 6000, 8000, 10000]



Discussion

Insertion

Observation: In both graphs, AVL insertion is consistently and noticeably *slower* than BST insertion across all data sizes. The gap widens as N increases by 10,000 nodes (Image 1), AVL takes roughly **~0.072s** compared to BST's **~0.026s**, nearly 3× slower.

Validation: This directly validates the general fact that *AVL insertion is slower than BST due to* rotations. Every insert into the AVL tree requires traversing back up the tree to check balance factors and potentially performing single or double rotations. With random data (which is what the benchmark generates), the BST remains reasonably balanced on average, so its insertion cost is low without any rotation overhead. The linear-looking growth in both curves is consistent with O(n log n) total cost for n insertions, but AVL carries a higher constant factor due to balancing work.

Key point: However, this slowdown is the cost of the guarantee. With sorted or adversarial input data, BST insertion would degrade to O(n) per operation (building a linked list), while AVL would remain O(log n). The benchmark uses random names, which is a best-case scenario for BST, yet AVL's overhead is still predictable and manageable.

Search

Observation: Search results are more nuanced and show some variability, but the trend is clear: AVL search times are consistently lower and more stable than BST search times. For smaller sizes, BST search hovers around 0.00011–0.00016s while AVL stays tighter around 0.000075–0.00011s. For larger sizes, BST search reaches up to ~0.000255s with noticeable fluctuation, while AVL stays flatter around 0.00015s.

Validation: This strongly validates the claim that *AVL generally performs faster or equal in search since the tree stays balanced.* Because AVL guarantees a height of at most 1.44 $\log_2(n)$, every search traverses at most that many nodes. The unbalanced BST with random data has an average height close to log n too, but it is not strictly bounded; some subtrees can grow taller, producing the variability and occasional spikes visible in the BST search line. The BST's jagged search curve in Image 2 is a direct symptom of this inconsistency.

Key point: The spikiness of the BST search line is itself informative; it reflects the non-deterministic height of an unbalanced tree. AVL's smoother curve demonstrates the predictability that comes from strict balancing.

Deletion

Observation: In both graphs, *AVL deletion is consistently slower than BST deletion*. In Image 1, BST deletion peaks around 0.00019s while AVL deletion reaches ~0.00041s at 10,000 nodes. In Image 2, the gap is visible from the very first size tested.

Validation: This validates the fact that *AVL deletion involves balancing overhead*. After removing a node, AVL must walk back up the tree, recomputing heights and performing up to O(log n) rotations to restore balance. BST deletion, by contrast, simply reconnects pointers with no further work. For random data at these sizes, this overhead is consistent and real, though still small in absolute terms (sub-millisecond). Crucially, AVL deletion time grows slowly and predictably, whereas BST deletion would degrade severely on sorted or skewed input.

Found Count

Though not plotted as a curve, the benchmark code tracks found_count for both trees using identical search queries. The expectation and observed behaviour is that both trees find the same contacts, since they store the same data and search by the same key. This validates that the correctness of the search is identical across both structures; the difference is purely in speed and consistency.

The Effect of Random Data

Both graphs use randomly generated lowercase names, which represent a near-best-case distribution for an unbalanced BST. Despite this, AVL still shows consistently faster search, validating that even under favourable random conditions, BST performance is less predictable. On real-world data, such as contacts entered alphabetically (e.g., "Aaron", "Alice", "Bob"...), the BST would degrade toward a right-skewed chain, making all operations O(n). AVL would be entirely unaffected, maintaining O(log n) throughout. The benchmark results, though using random data, already hint at this advantage through the BST's search variability.

AVL's Height Guarantee

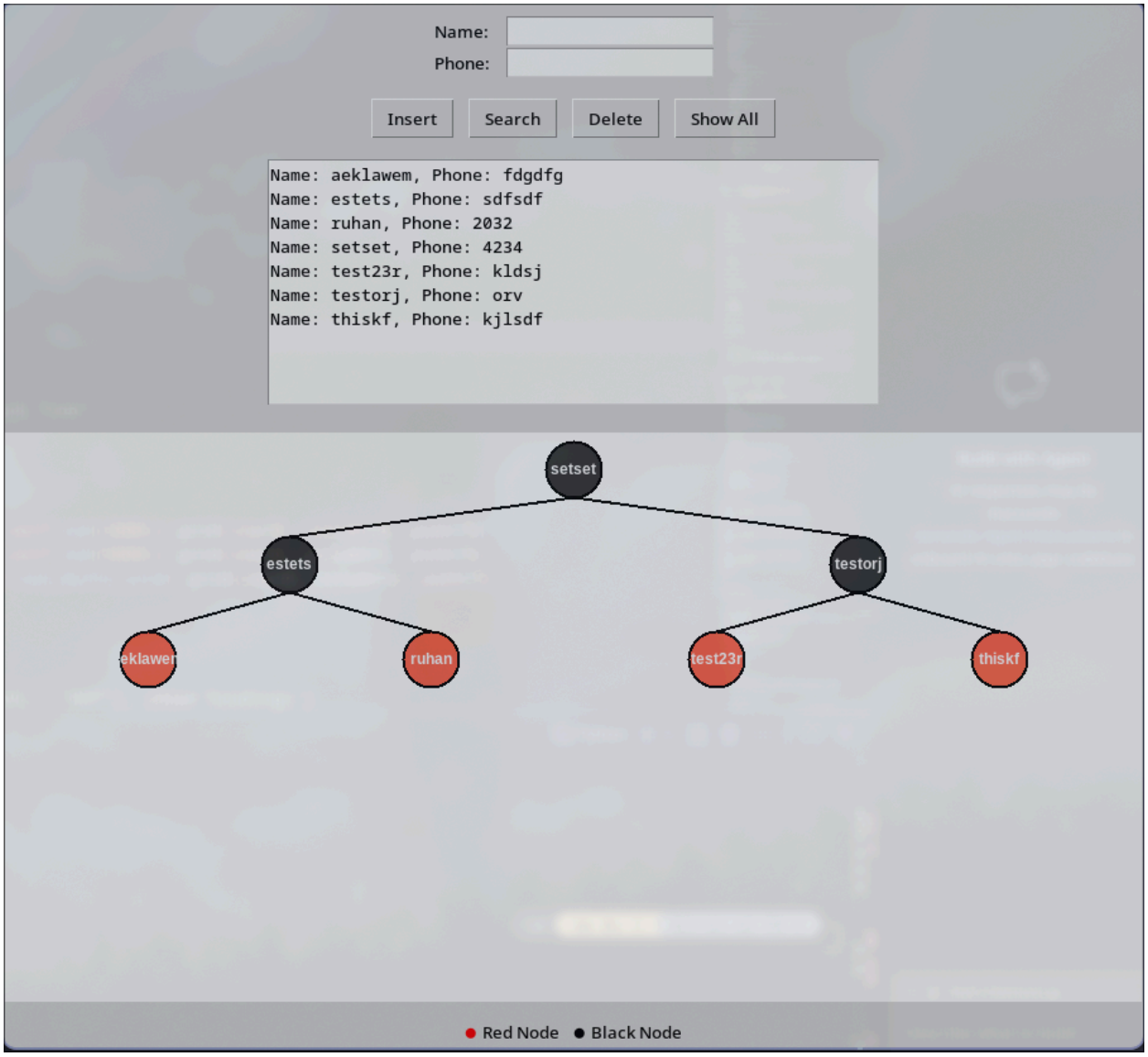
The graphs implicitly validate the height guarantee. AVL search and deletion both scale gently and smoothly from 500 to 10,000 nodes, consistent with O(log n) behaviour. The BST curves, while also roughly logarithmic under random data, fluctuate more, reflecting the absence of any structural guarantee. AVL's bounded height of $\leq 1.44 \log_2(n+2)$ is the mathematical foundation behind its stable performance curves.

Rotation Cost vs Benefit

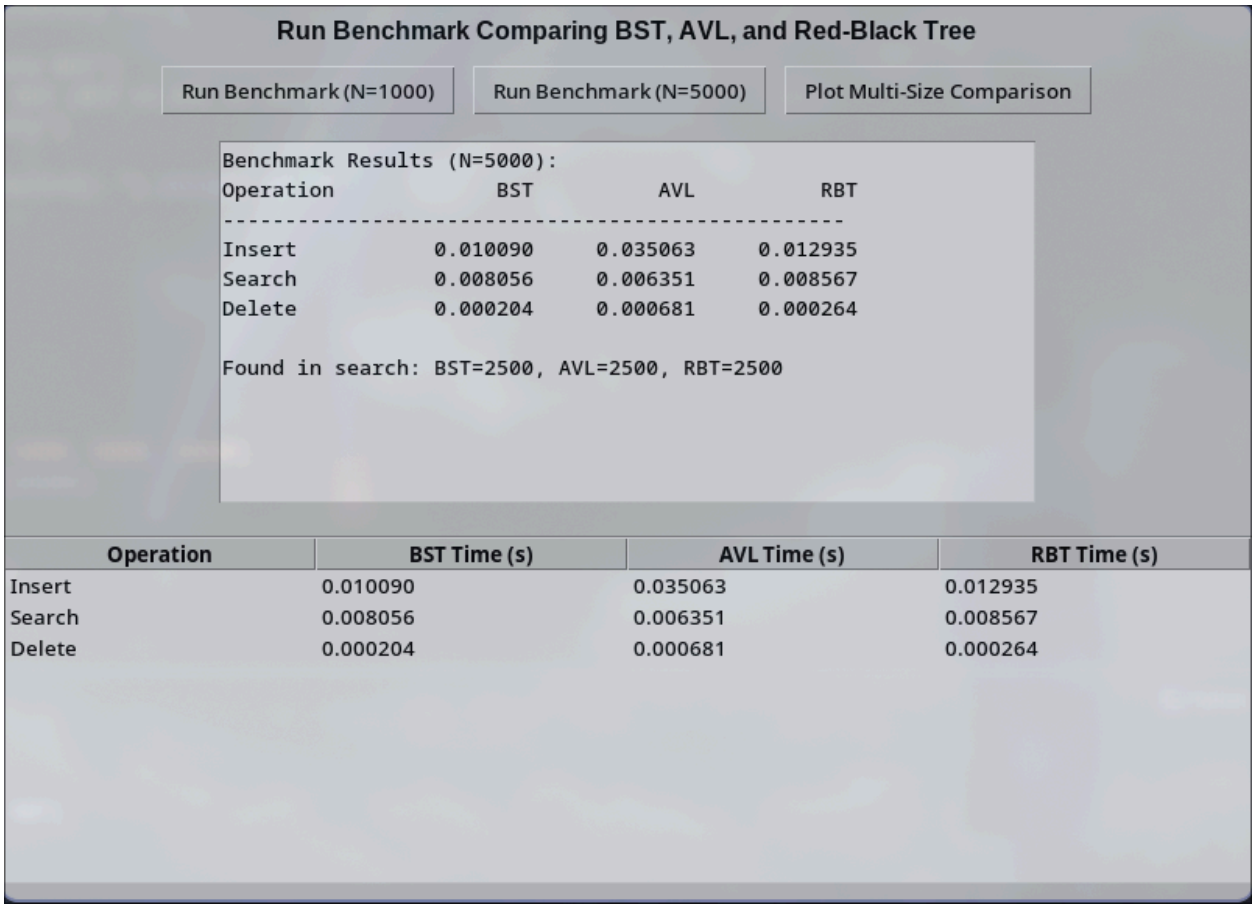
The insertion graphs show that AVL's rotation overhead is a real computational cost, roughly 2-3x insertion time compared to BST at large sizes. However, viewed against the benefit (guaranteed O(log n) search and deletion at all data sizes and orderings), this tradeoff is clearly worthwhile for large, dynamic, or non-random datasets. The deletion graph makes this even clearer: AVL's extra deletion cost is small in absolute terms (~0.0004s at n=10,000) while the guarantee it provides would prevent catastrophic O(n) degradation on skewed inputs.

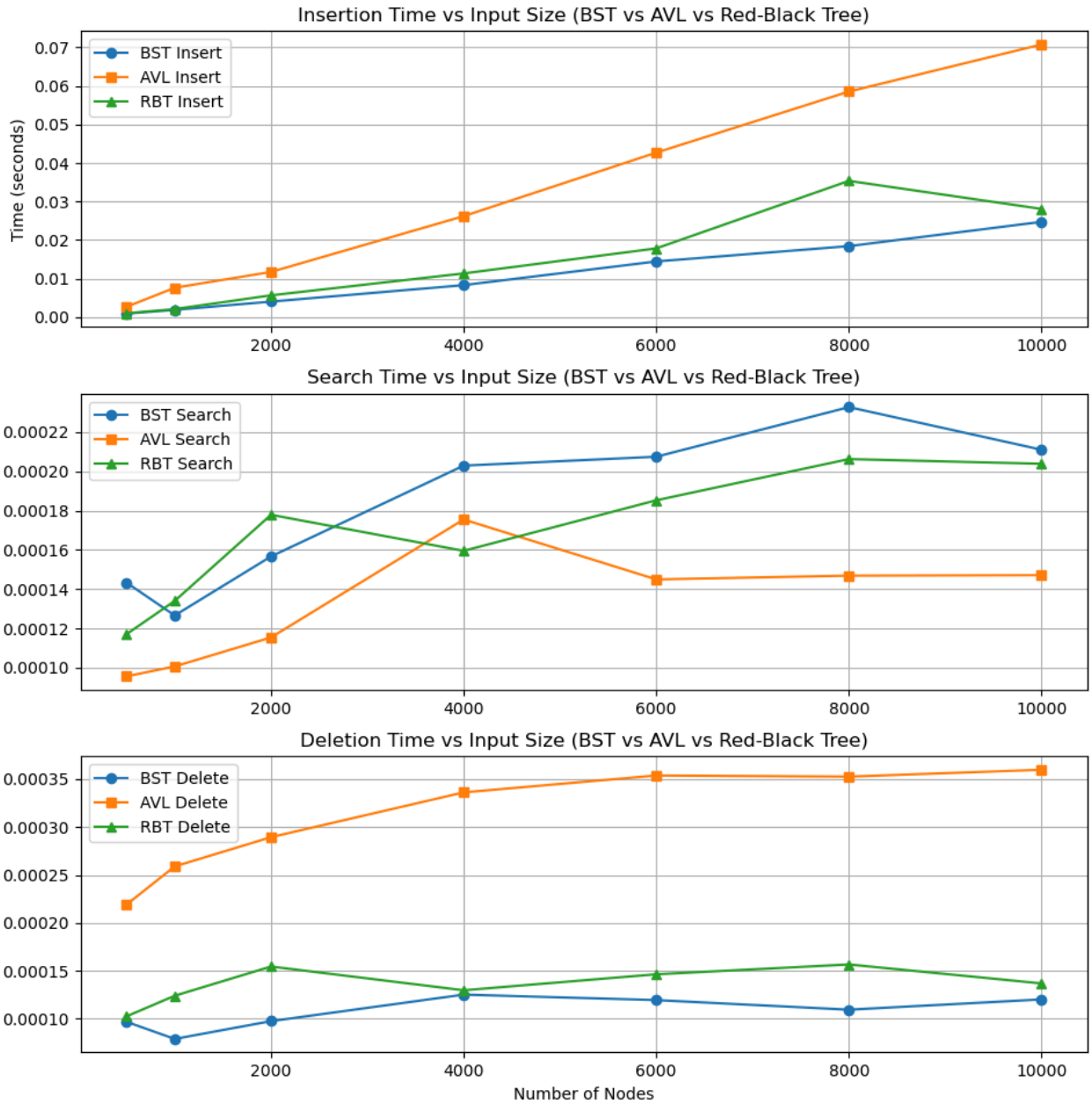
Task 5 - Design, Implement, Visualise and Benchmark the Red-Black-Tree

Implementation of Red-Black Tree



Benchmark BST vs AVL vs Red-Black Tree Benchmark



Operation	BST Time (s)	AVL Time (s)	RBT Time (s)
Insert	0.010090	0.035063	0.012935
Search	0.008056	0.006351	0.008567
Delete	0.000204	0.000681	0.000264

Visualisation of Benchmark